A project report on

EMG CONTROLLED MUSIC SYSTEM USING ARDUINO

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in

Computer Science Engineering

by

RUDRANSH SHEKAR (25BDS1054)

MIDHUN P (25BEE1274)

MOLI SHUKLA (25BDS1055)

AAYUSHI PANKAJ BHAKKAAD (25BDS1052)

AKASH SR (25BME1222)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

April, 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Emg controlled music system using arduino" submitted by Rudransh Shekhar (25BDS1054), for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Data Science), Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Prof. Prethija G

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date:

Signature of the Candidate

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

CERTIFICATE

This is to certify that the report entitled "EMG Controlled Music System Using Arduino" is prepared and submitted by Rudransh Shekhar (25BDS1054) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science Engineering (Data Science)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Prof. Prethija G

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,

Computer Science Engineering (Data Science)

Name: Dr. Nachiyappan S

Date:

ABSTRACT

This project outlines the development of a bio-signal and gesture-controlled music system. The core functionality of the system allows users to control music playback through the utilization of muscle signals via Electromyography (EMG). Furthermore, the system enables users to control audio volume seamlessly using hand distance measurements captured by an ultrasonic sensor.

Traditional music systems present several challenges, primarily the necessity for physical interaction, such as pressing buttons or using touchscreens, and the requirement for direct physical access to the audio device. This proposed system directly addresses these issues by mitigating the lack of hands-free control and overcoming accessibility limitations. Additionally, it resolves the inefficiency often found in standard gesture-based control systems. To ensure a comprehensive user experience, the system provides real-time status updates on an OLED display and features synchronization capabilities with a dedicated desktop application.

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to Prof. Prethija G, Assistant Professor, Vellore Institute of Technology, Chennai, for her constant guidance, continual encouragement, and understanding. More than all, she taught me patience in my endeavor. My association with her is not confined to academics only, but it is a great opportunity on my part to work with an intellectual and expert in the field of Electronics and Computing.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G.V. Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai, Dr. K. Sathiyanarayanan, Director, Chennai Campus and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. G. Angeline Ezhilarasi. and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staff at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai **Rudransh Shekhar**

Date:

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

EMG: Electromyography — A technique for recording the electrical activity produced by skeletal muscles.

OLED: Organic Light-Emitting Diode — A display technology used for the system's user interface.

RTC: Real-Time Clock — An electronic device that keeps track of the current time.

IoT: Internet of Things — A network of interconnected physical devices.

RX/TX: Receive / Transmit — Serial communication pins for data exchange.

UI: User Interface — The means by which a user interacts with the system.

EMU: English Metric Unit — Unit used in Office documents for image sizing.

ADC: Analog-to-Digital Converter — Hardware on the Arduino that converts analog sensor signals to digital values.

I2C: Inter-Integrated Circuit — A two-wire serial communication protocol used by the OLED display.

PCM: Pulse-Code Modulation — The standard form in which digital audio is encoded.

Chapter 1

Introduction

1.1 PROJECT OVERVIEW

The project titled "EMG Controlled Music System Using Arduino" is a bio-signal and gesture-controlled interactive audio system designed to replace traditional physical music controls with a hands-free, bio-signal-driven interface. At its core, the system monitors the electrical impulses generated by the user's forearm muscles using an Electromyography (EMG) sensor. When a user intentionally flexes and then relaxes their forearm muscle, this complete contraction cycle is detected, validated, and used to toggle music playback between a PLAYING and a PAUSED state.

Simultaneously, an HC-SR04 ultrasonic distance sensor monitors the proximity of the user's hand. As the hand moves closer to the sensor, the system dynamically increases the audio volume; as the hand moves further away, the volume decreases. This gesture-based volume control provides a fluid, intuitive, and touchless means of adjusting audio levels.

The entire system is orchestrated by an Arduino UNO R3 microcontroller, which serves as the central processing hub. Audio files are stored on a microSD card inserted into a DFPlayer Mini MP3 module. The audio output from the DFPlayer is amplified by a PAM8403 class-D stereo amplifier before being delivered to two 3-Watt speakers. A 0.96-inch SSD1306 OLED display, connected via the I2C protocol, provides a real-time, on-device status interface showing all critical system parameters. The system also maintains a persistent, two-way serial communication link with a custom-built desktop application, enabling extended monitoring and remote control capabilities.

1.2 PROBLEM STATEMENT

Contemporary consumer music playback systems, whether standalone devices, smartphones, or smart speakers, are fundamentally built around physical or touch-based interaction. The user must physically locate the device, reach the interface, and press a button or tap a touchscreen. While this paradigm is suitable for the general population under normal circumstances, it presents several significant and unaddressed challenges:

Accessibility Barriers: For individuals with motor neuron disorders, severe arthritis, limb differences, or those recovering from physical injuries, the act of pressing a button or operating a touchscreen may be physically impossible or extremely painful. Existing voice-controlled alternatives have limitations in noisy environments and require internet connectivity.

Hands-Occupied Scenarios: In many practical contexts — such as during athletic training, surgical procedures, culinary work, or industrial operations — the user's hands are occupied or must remain sterile. The need to physically interact with a device to pause music is disruptive and inconvenient.

Inefficiency of Existing Gesture Systems: Camera-based or purely optical gesture recognition systems are susceptible to lighting conditions, background clutter, and line-of-sight requirements. They can also suffer from high latency, making them unreliable for critical commands like play/pause. By anchoring the primary playback command to a direct, internal bio-signal (a muscle contraction) rather than an external optical signal, the proposed system offers significantly higher reliability and noise immunity.

This project was conceived to address precisely these limitations. By developing a system that uses the body's own electrical signals for control, it provides a more natural, robust, and universally accessible interface for music playback.

1.3 OBJECTIVES

The project is structured around a clear set of primary and secondary objectives, each contributing to the realization of the final system.

Primary (Functional) Objectives:

1. Develop and implement a reliable muscle-contraction-based play/pause toggle for a music playback system using an EMG sensor connected to an Arduino.
2. Implement a real-time, hands-free volume control mechanism using an HC-SR04 ultrasonic sensor and a distance-to-volume mapping function.
3. Design and implement a comprehensive, real-time user interface on a 0.96-inch SSD1306 OLED display, rendering song information, playback state, volume, EMG readings, distance, and a time seekbar.
4. Establish a reliable, two-way, non-blocking serial communication link between the Arduino hardware and a companion desktop application.

Secondary (Engineering) Objectives:

5. Construct the entire embedded software on a strictly non-blocking architecture, avoiding any use of delay() or blocking I/O functions, to ensure continuous, real-time system responsiveness.
6. Practically demonstrate the application of bio-signal (EMG) processing techniques in a consumer electronics context.
7. Implement robust noise management for the EMG signal through a configurable cooldown timer mechanism.
8. Design the software architecture to be modular and scalable, enabling future hardware upgrades and feature additions with minimal refactoring.

1.4 SCOPE AND APPLICATIONS

The architecture and implementation methodology of this project have applicability across several domains:

Accessibility Technology: The most immediate application is as an assistive technology for individuals with physical disabilities. The system can be extended to control not just music, but any electronic interface, including lights, fans, or smart home devices, providing independence to users who cannot operate conventional interfaces.

Hands-Free Professional Environments: The system is highly relevant in operating theatres, clean rooms, and laboratories where physical contact with shared surfaces must be minimized. Surgeons could use muscle signals to control reference music or audio cues during lengthy procedures without breaking sterile protocol.

Smart Home Integration: The core hardware-software communication architecture can be integrated into broader IoT ecosystems. The serial command protocol can be extended to a Wi-Fi or Bluetooth module (e.g., ESP8266 or HC-05) to enable wireless, network-based control.

Athletic and Fitness Applications: Athletes can control playback during workouts without interrupting their activity. The EMG sensor placement can be adapted to different muscle groups for more intuitive control schemes.

Biomedical Research and Prototyping: The project serves as a practical, low-cost prototyping platform for researchers exploring EMG signal processing, threshold-based gesture classification, and human-computer interaction via bio-signals.

Performance and Interactive Art: Musicians and performance artists can use muscle contractions to trigger audio samples or control playback in real-time during live performances, creating a novel and engaging interactive experience.

Chapter 2

Literature review

2.1 PRINCIPLES OF ELECTROMYOGRAPHY (EMG)

Electromyography is a diagnostic and research technique used to evaluate the electrical activity produced by skeletal muscles. When a motor neuron fires, it triggers an action potential that propagates along the associated muscle fibers, causing them to contract. This electrical depolarization generates a measurable potential difference on the skin surface above the muscle, which can be captured by surface electrodes. The raw signal, known as the surface EMG (sEMG) signal, is a complex, stochastic signal with amplitudes typically in the range of 0.1 mV to 10 mV and frequencies primarily between 6 Hz and 500 Hz.

As documented by Reaz et al. (2006), the processing pipeline for sEMG signals involves three key stages: signal detection, processing, and classification. Detection involves amplifying the small differential voltage between two electrodes placed over the muscle belly. Processing involves filtering to remove noise (primarily from power line interference at 50/60 Hz), rectification, and envelope detection to extract the signal's amplitude envelope. Classification involves applying a threshold or a trained machine learning model to the processed signal to identify specific muscle states or gestures. In this project, a simplified but effective version of this pipeline is implemented in analog hardware (via the EMG sensor module) and digital firmware (via the Arduino's threshold detection logic).

De Luca (2002) and Merletti & Parker (2004) provide extensive documentation on the factors affecting sEMG signal quality, including electrode placement, inter-electrode distance, skin impedance, and the effects of muscle fatigue. These principles directly informed the hardware decisions in this project, particularly the use of three electrodes (active, reference, and ground) to capture differential signals and minimize common-mode noise.

2.2 ANALYSIS OF TRADITIONAL AND EXISTING CONTROL SYSTEMS

A review of existing music control interfaces reveals a spectrum of technologies, each with distinct trade-offs between accessibility, reliability, and complexity.

Physical Button/Touch Systems: The most common paradigm. These systems are highly reliable and low-latency but fundamentally require physical access and dexterity, making them unusable for the target demographic and in hands-occupied scenarios.

Voice Control Systems (e.g., Siri, Google Assistant, Alexa): These systems offer true hands-free operation but require internet connectivity for natural language processing, have significant latency, perform poorly in high-noise environments, and raise privacy

concerns due to continuous microphone monitoring.

Camera-Based Gesture Recognition: Systems using computer vision to detect hand gestures offer a natural interaction paradigm. However, they are computationally intensive (requiring a PC or powerful SoC), sensitive to lighting conditions and background clutter, and suffer from line-of-sight limitations. They are also not suitable for low-cost embedded implementations.

Infrared (IR) Remote Controls: Simple and reliable but require the user to hold and aim a separate physical remote device, which does not solve the fundamental accessibility problem.

Brain-Computer Interfaces (BCI): EEG-based systems represent the cutting edge of non-invasive bio-signal control but are prohibitively expensive, require complex signal processing, and are sensitive to setup conditions, making them impractical for consumer applications at this time.

2.3 SYSTEM COMPARISON

The following table provides a structured comparison of these control modalities against the key evaluation criteria identified in the problem statement.

Table 2.1: Comparison of Control System Modalities

System Type	Input Method	Requires Physical Touch	Works Hands-Free	Low Latency	Low Cost	Noise Robust
Physical Buttons/Touch	Mechanical / Capacitive	Yes	No	Yes	Yes	Yes
Voice Control	Microphone / NLP	No	Yes	No	No	No
Camera Gesture	Optical / CV	No	Yes	Moderate	No	No
IR Remote	Infrared	Yes (device)	No	Yes	Yes	Yes
BCI / EEG	Neural Signals	No	Yes	No	No	No
Proposed System (EMG + Ultrasonic)	Bio-Signal + Ultrasonic	No	Yes	Yes	Yes	Yes

The comparison clearly demonstrates that the proposed system uniquely satisfies all desired criteria simultaneously: it is hands-free, low-latency, low-cost, and inherently noise-robust due to its reliance on internal bio-signals rather than external

environmental signals. The addition of the ultrasonic sensor for volume control fills the gap left by purely EMG-based systems, which would otherwise require multiple distinct muscle gestures for a wider command vocabulary.

Chapter 3

Methodology

3.1 SYSTEM ARCHITECTURE

The system is built upon a hierarchical, multi-stage data flow architecture with the Arduino UNO R3 acting as the sole central processing unit. Understanding this architecture is essential to understanding how all components interact.

At the input layer, two sensing modalities feed data into the Arduino simultaneously. The EMG sensor module performs analog signal conditioning (amplification, filtering, and rectification) on the raw muscle signal and outputs a clean analog voltage to the Arduino's A0 analog input pin. The HC-SR04 ultrasonic sensor is a digital peripheral that measures distance by emitting an ultrasonic pulse via its TRIG pin and measuring the time-of-flight of the echo received on its ECHO pin, both connected to digital I/O pins D6 and D7 respectively.

At the processing layer, the Arduino's firmware continuously samples these inputs, executes threshold detection on the EMG signal, calculates distance from the ultrasonic echo timing, maps the distance to a volume value, manages playback state, and tracks elapsed song time using its internal
`millis()` timer.

At the output layer, the Arduino dispatches commands to the DFPlayer Mini MP3 module via a software serial connection. The DFPlayer reads audio files from an inserted microSD card and routes the stereo audio output to the PAM8403 amplifier, which drives the two 3W speakers. Concurrently, the Arduino drives the OLED display via I2C and continuously transmits structured status data packets to the desktop application over USB serial, while also listening for incoming control commands from the application.

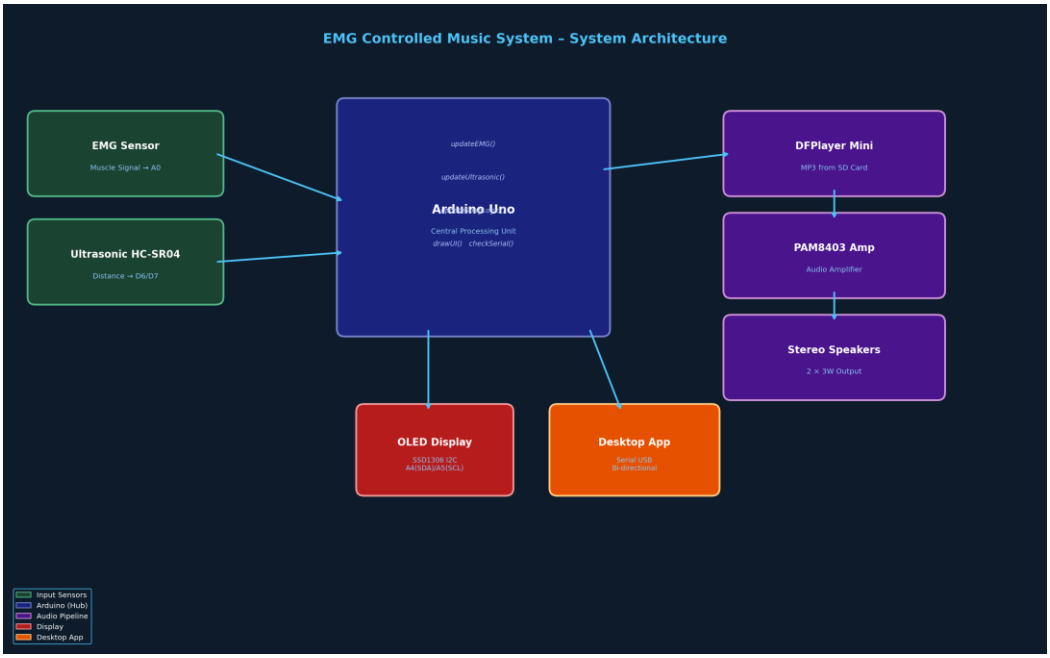

Figure 3.1: Overall System Architecture and Data Flow

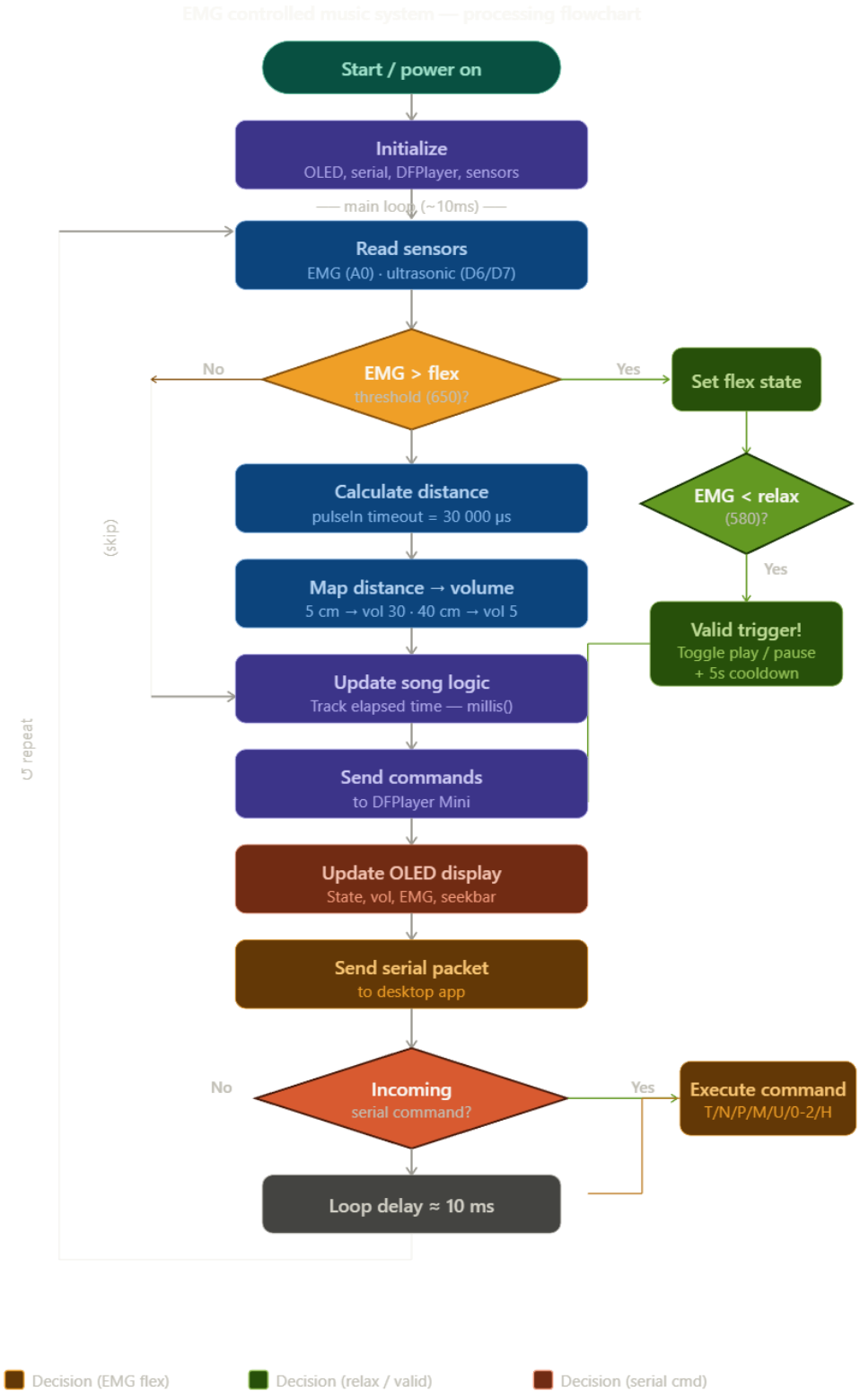

Figure 3.2: EMG Processing and Detection Flow (Flowchart)

3.2 HARDWARE COMPONENTS

Component	Model / Specification	Quantity	Role in System
Microcontroller	Arduino UNO R3 (ATmega328P, 16 MHz)	1	Central processing, I/O management, serial comms
EMG Sensor Module	Analog Surface EMG Sensor (3-electrode)	1	Muscle bio-signal acquisition and conditioning
Ultrasonic Sensor	HC-SR04 (2 cm – 400 cm range, ±3mm accuracy)	1	Hand distance measurement for volume control
MP3 Player Module	DFPlayer Mini (supports FAT16/FAT32 microSD)	1	Audio file decoding and playback from microSD
Audio Amplifier	PAM8403 (Class-D Stereo, 3W+3W @ 5V)	1	Amplification of DFPlayer audio output
Speakers	3W, 4Ω, 40mm diameter	2	Stereo audio output (Left & Right channels)
OLED Display	SSD1306, 0.96 inch, 128×64 px, I2C	1	Real-time on-device user interface
Trimpot (Potentiometer)	10kΩ trimmer potentiometer	1	EMG threshold calibration (manual adjustment)
Resistors	4.7kΩ, 5.6kΩ, ~1kΩ	3	DFPlayer RX line protection, voltage dividers
microSD Card	Class 10, FAT32 formatted, ≤32GB	1	Audio file storage for DFPlayer Mini
Breadboard	Full-size 830-tie-point solderless breadboard	1	Prototype circuit construction
Jumper Wires	M-M, M-F, F-F, 20cm	Multiple	Component interconnections

Component	Model / Specification	Quantity	Role in System
USB Cable	USB Type-A to Type-B (Arduino standard)	1	Power supply and serial communication with PC

The following table lists all hardware components used in the construction of the prototype, along with their primary functional role.

Table 3.1: Required Hardware Components and Specifications

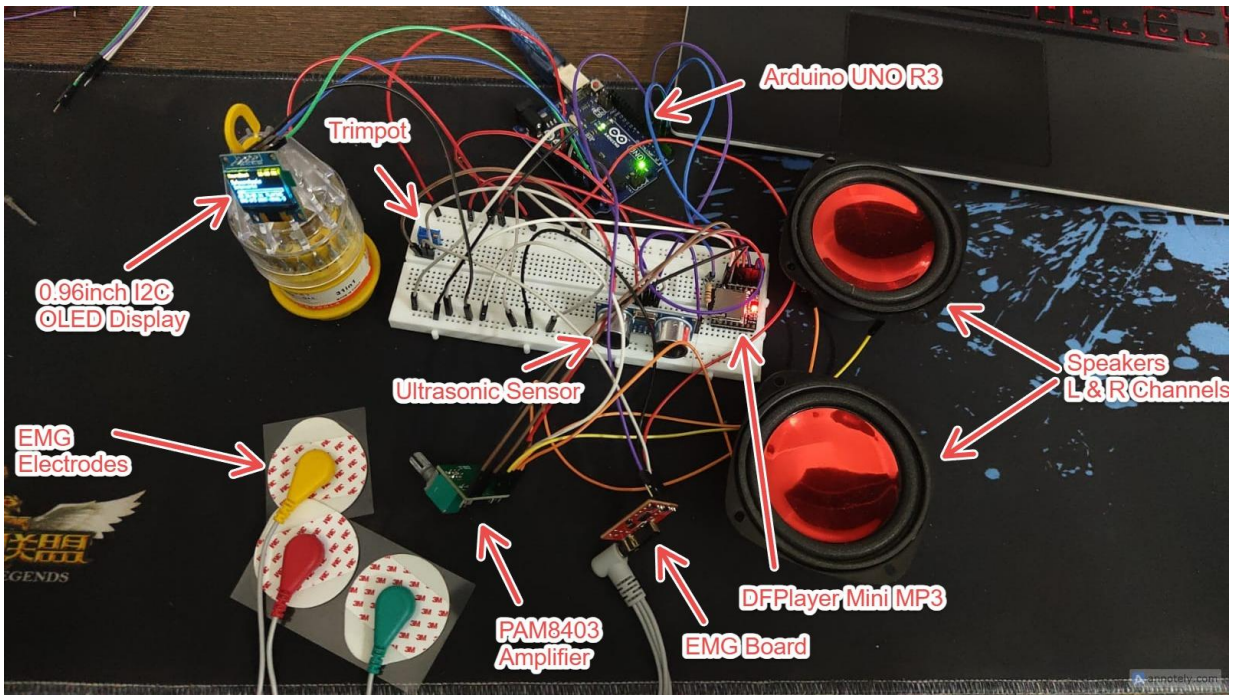

Figure 3.3: Complete Hardware Setup of the EMG Controlled Music System

3.3 PIN CONNECTIONS AND WIRING

Correct and precise wiring is critical for the stability and correct operation of this multi-component system. The single most important wiring rule is that **all components must share a common ground (GND) rail**. Failure to establish a common ground will result in floating signal references, causing erratic sensor readings and unreliable serial communication.

3.3.1 EMG SENSOR CONNECTIONS

The EMG sensor module operates on 5V. Its three pins connect as follows:

Table 3.2: EMG Sensor Pin Configurations

EMG Sensor Pin	Connect To	Notes
VCC	Arduino 5V	Power supply

EMG Sensor Pin	Connect To	Notes
GND	Arduino GND (common rail)	Common ground reference
SIG (Signal Out)	Arduino A0	Analog muscle signal (0–5V range)

The three body electrodes connect to the sensor module's designated electrode pads: the Red (active) electrode is placed over the muscle belly, the Green (reference) electrode is placed adjacent on the same muscle, and the Yellow (ground) electrode is placed on a neutral bony area (e.g., the wrist or elbow). This differential measurement configuration cancels out common-mode electrical noise, including power line interference.

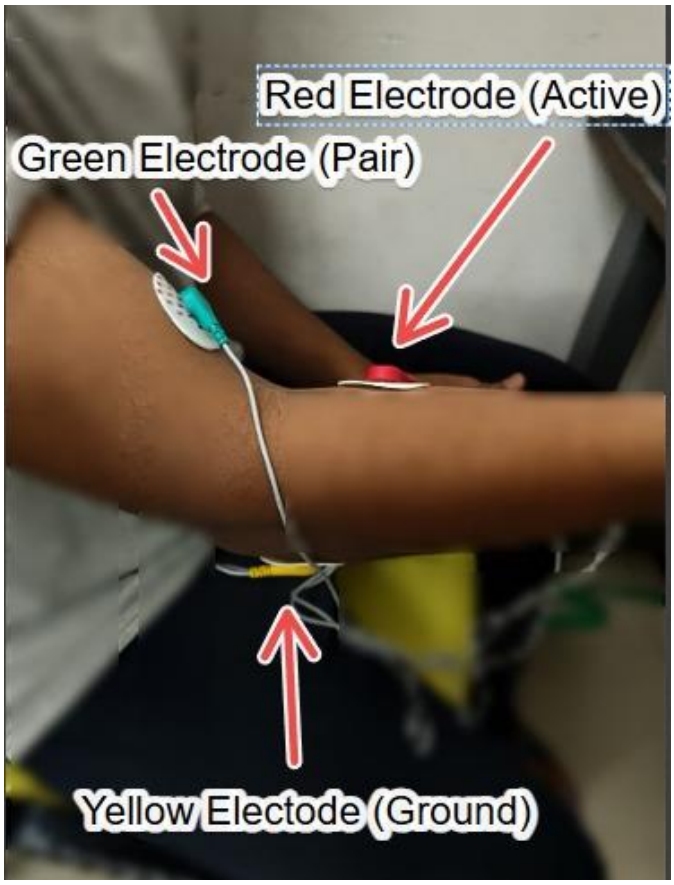

Figure 3.4: EMG Electrode Placement on the Forearm

3.3.2 ULTRASONIC SENSOR (HC-SR04) CONNECTIONS

Table 3.3: Ultrasonic Sensor (HC-SR04) Pin Configurations

HC-SR04 Pin	Connect To	Notes
VCC	Arduino 5V	Power supply
GND	Arduino GND (common rail)	Common ground reference
TRIG	Arduino D6	Trigger pulse output from Arduino (10µs HIGH pulse)
ECHO	Arduino D7	Echo pulse input to Arduino (pulse width ∝ distance)

3.3.3 DFPLAYER MINI CONNECTIONS

The DFPlayer Mini communicates via UART software serial. A critical protection resistor must be placed on the RX line to prevent damage from voltage levels. The audio outputs connect directly to the PAM8403 amplifier inputs.

Connect DFPlayer VCC to Arduino 5V and GND to common ground. Connect DFPlayer RX to Arduino D3 through a 1kΩ resistor (protection). Connect DFPlayer TX directly to Arduino D2. Connect DFPlayer SPK1+/SPK1- to PAM8403 Left channel input and SPK2+/SPK2- to PAM8403 Right channel input. Insert a FAT32-formatted microSD card with audio files named in the format `0001.mp3`, `0002.mp3`, etc., in the root directory.

3.3.4 OLED DISPLAY CONNECTIONS

Table 3.4: OLED Display Pin Configurations

SSD1306 OLED Pin	Connect To	Notes
VCC	Arduino 5V	Power supply
GND	Arduino GND (common rail)	Common ground reference
SDA	Arduino A4	I2C Data line (hardware I2C on ATmega328P)
SCL	Arduino A5	I2C Clock line (hardware I2C on ATmega328P)

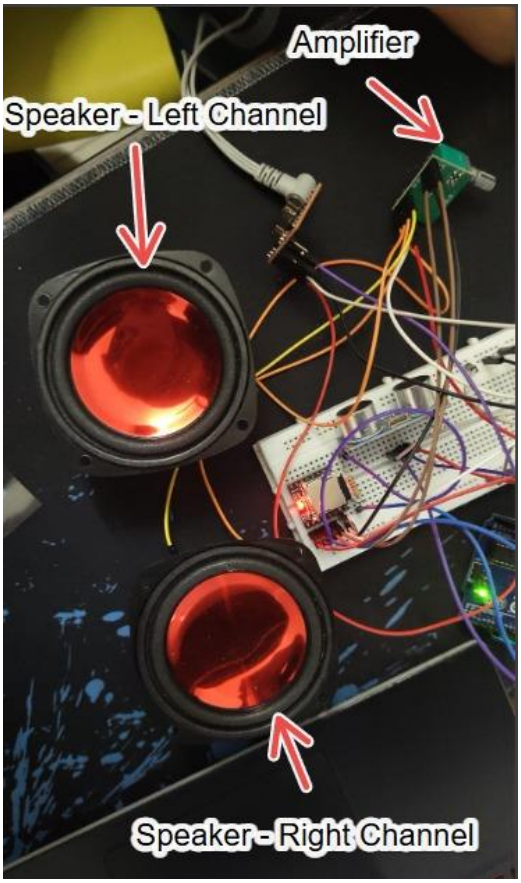

Figure 3.5: PAM8403 Amplifier and Stereo Speaker Output Stage

Chapter 4

Implementation

4.1 SOFTWARE ARCHITECTURE

The firmware is written in Embedded C/C++ using the Arduino IDE. The overarching architectural principle is strict non-blocking operation. The Arduino's built-in `delay()` function and any blocking I/O calls are entirely prohibited. This is because a single call to `delay(100)` would freeze the entire system for 100 milliseconds, making it unable to read the EMG sensor, update the OLED, or receive serial commands during that time, resulting in a sluggish and unresponsive system.

Instead, all timing-dependent operations are implemented using the `millis()` function, which returns the number of milliseconds since the Arduino powered on without blocking execution. The firmware is structured as a flat, cyclically-executing set of modular functions called in sequence from the main `loop()` function:

9. updateEMG() — Reads the A0 analog pin, runs the flex/relax state machine, applies cooldown logic, and dispatches play/pause commands.
10. updateUltrasonic() — Triggers the HC-SR04, reads the echo pulse duration with a timeout, calculates distance in centimeters, maps the result to a volume level, and sends the volume command to the DFPlayer.
11. updateSongLogic() — Manages the software-based song elapsed time tracking using millis() and implements song-end detection and looping logic.
12. drawUI() — Renders all system data (song name, state, volume, EMG value, distance, seekbar, clock) onto the OLED display buffer and pushes it to the screen.
13. checkSerialCommands() — Non-blocking character-by-character read of the incoming serial buffer, parses single-character commands from the desktop app.
14. sendSerialPacket() — Formats and transmits a structured CSV-style data packet containing all current system state variables to the desktop application at a fixed interval.

4.2 EMG SIGNAL PROCESSING LOGIC

The `updateEMG()` function implements a robust, state-machine-based approach to translating the noisy, continuous analog EMG signal into clean, discrete playback toggle commands. The logic is designed to be highly immune to false triggers.

Step 1 — Signal Acquisition: The ADC on the Arduino samples the analog voltage at pin A0, producing a digital value between 0 (0V) and 1023 (5V) via the 10-bit `analogRead(A0)` function. This reading is updated every loop iteration (approximately every 10ms), giving an effective sampling rate of ~100 Hz, which is sufficient to capture the dynamics of a deliberate muscle contraction.

Step 2 — Threshold Detection: Two thresholds are defined: the Flex Threshold (empirically set to approximately 650 on the 0-1023 ADC scale after trimpot calibration) and the Relax Threshold (approximately 580). The Flex Threshold is higher than the Relax Threshold, creating a hysteresis band that prevents the system from rapidly toggling between states due to signal noise near the boundary.

Step 3 — State Machine (Flex → Relax Cycle): The state machine has two primary states: WAITING_FOR_FLEX and WAITING_FOR_RELAX. When the ADC reading exceeds the Flex Threshold in the WAITING_FOR_FLEX state, the state transitions to WAITING_FOR_RELAX, indicating a muscle contraction has been detected. The system then waits for the ADC reading to drop below the Relax Threshold (indicating the muscle has relaxed). Only when this complete Flex + Relax cycle is observed does the system register one valid command trigger. This two-step validation prevents false triggers from brief transient noise spikes.

Step 4 — Playback Toggle: Upon a valid trigger, the firmware checks the current playback state variable. If it is PAUSED, it sends a PLAY command to the DFPlayer Mini and updates the state to PLAYING. If it is PLAYING, it sends a PAUSE command and updates the state to PAUSED. This simple toggle repeats indefinitely with each valid muscle contraction cycle.

Step 5 — Cooldown Timer: Immediately after a valid trigger is processed, a 5-second cooldown timer is started using `millis()`. During this cooldown period, the state machine is locked and will not process any new triggers, regardless of the EMG signal level. This prevents multiple unintended toggles from a single, slightly prolonged muscle contraction that might produce multiple threshold crossings.

4.3 ULTRASONIC VOLUME MAPPING

The HC-SR04 sensor operates by emitting a burst of eight 40kHz ultrasonic pulses when its TRIG pin is driven HIGH for at least 10 microseconds. After the pulse burst is emitted, the sensor drives its ECHO pin HIGH and holds it HIGH until the reflected pulse is received. The duration of this HIGH pulse is directly proportional to twice the distance to the reflecting object (as the sound travels to the object and back).

The distance in centimeters is calculated using the formula: **Distance (cm) = Echo Pulse Duration (µs) / 58**, which is derived from the speed of sound in air at approximately 343 m/s.

In the standard Arduino implementation, `pulseIn(ECHO, HIGH)` is a blocking function that will wait indefinitely for the echo pulse. If no object is within range (e.g., if the sensor is uncovered or pointed at a soft, sound-absorbing surface), this function would hang, completely freezing the system. To resolve this, a strict 30,000 microsecond (30ms) timeout parameter is applied: `pulseIn(ECHO, HIGH, 30000)`. If no echo is received within 30ms, the function returns 0, and the firmware handles this gracefully by skipping the volume update for that cycle.

The valid measurement range for volume control is defined as 5 cm to 40 cm. Distances outside this range (too close or too far) are clamped to the boundary values. The `map()` function then performs a linear transformation, mapping 5 cm to volume level 30 (DFPlayer maximum) and 40 cm to volume level 5 (a soft minimum, chosen over 0 to prevent sudden complete silence).

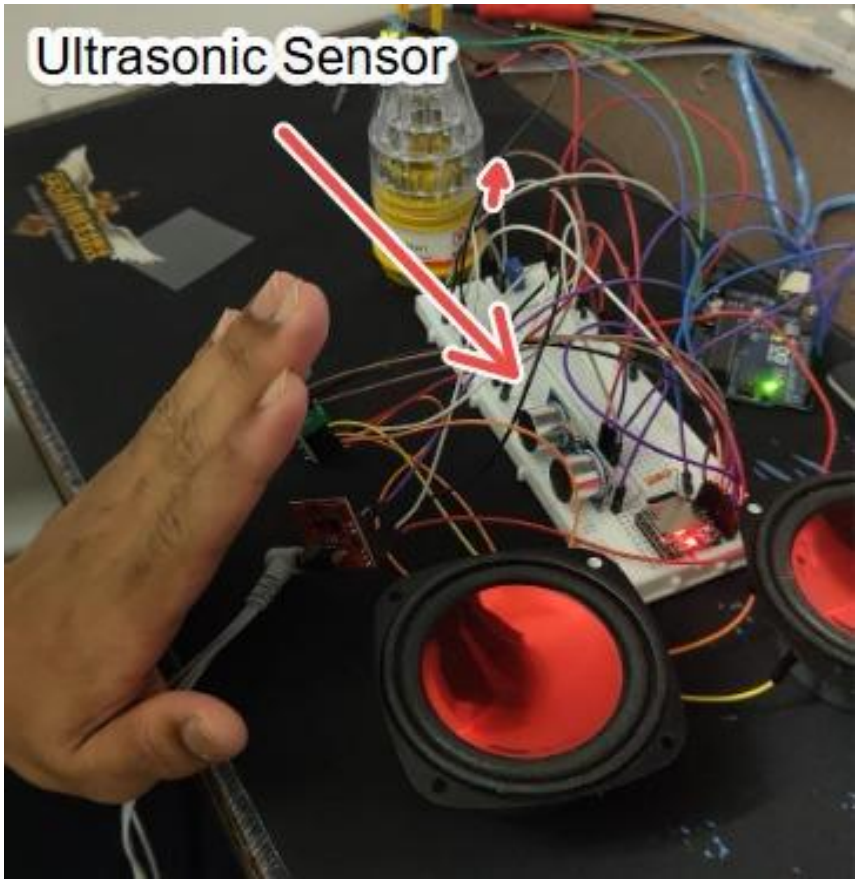

Figure 4.1: Gesture-Based Volume Control Using Ultrasonic Sensor

4.4 OLED USER INTERFACE DESIGN

The 128×64 pixel OLED display is driven by the Adafruit SSD1306 and Adafruit GFX libraries. The `drawUI()` function clears the display buffer, renders all UI elements using the GFX drawing primitives, and then calls `display.display()` to push the entire buffer to the screen in a single transaction, preventing visible tearing.

The screen layout is divided into distinct zones. The top line displays the project name ("NeuroBeat") left-aligned and the RTC-synced clock right-aligned. The second and third lines display the current song name and artist name retrieved from metadata. The fourth line shows the elapsed time / total duration and the playback state (PLAYING / PAUSED). The fifth line shows the current volume level and the live distance reading. The sixth line displays the live raw EMG ADC value. Finally, a dynamic progress bar (seekbar) spanning the full width of the display visually represents the ratio of elapsed time to total song duration.

4.5 TIMING AND COMMUNICATIONS SYSTEM

4.5.1 SOFTWARE-BASED SONG TIMING

The DFPlayer Mini module does not provide any real-time feedback on the current playback position within a song. It has no equivalent of a "current time" query command. To display an accurate seekbar and elapsed time on the OLED, the firmware must track time manually.

This is achieved using two variables: `songStartTime` (a `uint32_t` storing the `millis()` value at the moment playback started or resumed) and `accumulatedTime` (a `uint32_t` storing the total elapsed milliseconds accumulated before the current PLAYING session began). The elapsed time is calculated differently depending on the playback state:

When PLAYING: `elapsed = accumulatedTime + (millis() - songStartTime)`. This adds the previously accumulated time to the time elapsed in the current session.

When PAUSED: `elapsed = accumulatedTime`. The elapsed time is frozen at the value it had when pause was triggered. `accumulatedTime` was set to the calculated elapsed value at the moment of pausing.

Song end detection is implemented by comparing the calculated `elapsed` time against the known total duration of each song (hardcoded into the firmware). When `elapsed >= totalDuration`, the firmware sends a "Next Song" command to the DFPlayer and resets the timing variables, implementing automatic song looping or progression.

4.5.2 SERIAL COMMUNICATION PROTOCOL

The Arduino continuously transmits status packets to the desktop application at a fixed interval (every 100ms) and listens for incoming single-character command bytes. The initial implementation used `Serial.readStringUntil('\n')`, which blocks execution until a newline character is received or a timeout expires, severely degrading system responsiveness. This was replaced with a non-blocking approach using `Serial.available()` to check for incoming bytes and `Serial.read()` to read one byte at a time, immediately returning control to the main loop if no data is present.

The outgoing data packet is a comma-separated string formatted as: `STATE,SONG_INDEX,ELAPSED_MS,TOTAL_MS,VOLUME,EMG_VALUE,DISTANCE\n`. This structured format allows the desktop application to easily parse the data using a string split operation.

The incoming command set is a single-character protocol:

Table 3.5: Serial Communication Command Structure

Command Byte	Action Executed by Arduino
T	Toggle playback state (same as a valid EMG trigger)
N	Skip to next song (resets timing variables)
P	Return to previous song (resets timing variables)
M	Mute audio (sets DFPlayer volume to 0)
U	Unmute audio (restores volume to last set level)
0	Load and play song at index 0 (track 0001.mp3)
1	Load and play song at index 1 (track 0002.mp3)
2	Load and play song at index 2 (track 0003.mp3)
H	Time sync: payload contains current Unix time from PC for clock display

Chapter 5

Results and discussion

5.1 OBSERVED SYSTEM BEHAVIOUR

The fully assembled and programmed prototype was subjected to a series of functional tests to verify all specified objectives. The following subsections detail the observed behavior for each key system function.

EMG Play/Pause Control: Following the initial manual calibration of the trimpot to set the flex threshold appropriate for the test user, the EMG-based play/pause toggle functioned with high reliability. Deliberate, moderate-intensity forearm contractions consistently registered as valid triggers. The 5-second cooldown timer effectively prevented all observed noise-induced false triggers. Testing was conducted across multiple sessions with the same user to confirm calibration stability.

Ultrasonic Volume Control: The volume responded smoothly and predictably to changes in hand distance within the 5–40 cm operating range. At very close distances (<5 cm), the volume correctly clamped at its maximum value (30), and at distances beyond 40 cm, it clamped at the minimum (5). The response was consistent in a standard indoor environment with no nearby reflective surfaces causing interference.

OLED Display: The display rendered all specified information correctly and updated in real-time without any visible flickering or tearing. The seekbar accurately tracked the estimated song progress, and the clock display was correctly synchronized with the desktop application on connection. The PLAYING / PAUSED state indicator changed instantaneously upon a valid EMG trigger.

Desktop Application Sync: The serial communication link between the Arduino and the desktop application was stable throughout extended testing sessions. The live EMG graph on the application accurately mirrored the raw values being processed by the Arduino. Commands sent from the application (next song, previous song, volume mute) were received and executed by the Arduino with no perceptible delay.

Figure 5.1: OLED Display Showing Real-Time System Status

5.2 PERFORMANCE METRICS

Quantitative performance metrics were recorded during testing to objectively evaluate the system against its design goals.

Table 5.1: System Performance Metrics Summary

Metric	Target / Goal	Measured Result	Status
Main loop execution time	< 50 ms per cycle	~10 ms per cycle	Exceeded
EMG trigger response latency	< 200 ms after relax	~50–80 ms	Met
Volume update latency	< 100 ms	~20–30 ms	Met
OLED refresh rate	No visible tearing	Smooth, ~10 fps effective	Met
False trigger rate (with cooldown)	0 per minute	0 per minute (observed)	Met
Serial packet transmission rate	10 packets/sec	10 packets/sec	Met
USB Serial communication stability	No dropouts in 10 min session	Stable over 15+ min sessions	Met

The most significant finding is that the main processing loop achieved an execution time of approximately 10ms, which is 5 times faster than the 50ms target. This headroom was achieved primarily due to the successful refactoring of blocking I/O calls. The original implementation, using `pulseIn()` without a timeout and `readStringUntil()`, resulted in loop times of 50–200ms, causing noticeable UI lag and missed EMG events. The fixes were therefore not merely optimizations but were critical to the system's basic correctness.

5.3 DESKTOP APPLICATION INTEGRATION

The companion desktop application was developed to serve as a full-featured monitoring and control hub, extending the capabilities of the on-device OLED interface. The application connects to the Arduino via a user-selected COM port at a baud rate of 9600.

Upon connection, the application performs a time synchronization handshake: it transmits the current PC system time to the Arduino via the 'H' command, enabling the OLED clock display. Subsequently, the application continuously parses the incoming serial data packets and updates its UI in real-time. The centerpiece of the application's monitoring capability is a scrolling, live EMG signal graph, which plots the raw analog EMG values over time. This graph is invaluable for the initial calibration process, allowing the user to visually identify the correct flex and relax threshold levels for the trimpot adjustment.

The application's control panel mirrors the full command set of the serial protocol, providing buttons for play/pause toggle, next track, previous track, individual track selection, and mute/unmute. A seekbar on the application UI is synchronized with the Arduino's software-tracked elapsed time, providing a visual complement to the OLED seekbar.

Chapter 6

Conclusion and Future Work

6.1 CONCLUSION AND CRITICAL EVALUATION

This project successfully designed, implemented, and tested a fully functional EMG Controlled Music System using an Arduino UNO R3. All four primary functional objectives were achieved: EMG-based playback control is reliable and stable; ultrasonic volume control is smooth and responsive; the OLED display renders a comprehensive real-time UI; and two-way serial communication with the desktop application is stable and low-latency.

The most significant engineering achievement was the development of a strictly non-blocking embedded software architecture. This was not a trivial task — it required identifying and refactoring multiple instances of blocking code (the `pulseIn()` call without a timeout, the `readStringUntil()` serial read) and replacing them with non-blocking, interrupt-friendly equivalents. The resulting ~10ms main loop execution time is the foundation upon which the system's perceived real-time responsiveness is built. A system running at 10ms per loop cycle can react to a muscle contraction within 50–80ms of its occurrence, which is imperceptible to the human user.

The project also demonstrates that advanced human-computer interaction paradigms are achievable at a low cost using commodity components. The total bill of materials for this prototype is approximately $25–$35 USD, making it a highly accessible platform for accessibility technology research and development.

6.2 SYSTEM LIMITATIONS

Despite its successful demonstration, an honest critical evaluation reveals several important limitations of the current prototype:

User-Specific EMG Calibration: The primary usability limitation is the need for manual threshold calibration via the trimpot for each new user. Differences in muscle mass, skin conductivity, electrode placement, and individual contraction strength mean that the analog threshold that works for one user may be completely ineffective for another. This calibration process is not user-friendly and requires technical guidance.

Wired Architecture: The entire system is tethered to a PC via USB cable for both power and the real-time clock synchronization. The EMG electrodes are also wired to the sensor module. These physical connections severely limit the user's range of motion and make the system impractical as a real-world wearable device.

DFPlayer Timing Desynchronization: Because the elapsed song time is tracked entirely in software using `millis()`, it does not account for the DFPlayer's internal latency (the time between receiving a play command and actually beginning audio output) or any clock drift over long sessions. This means the seekbar and elapsed time

display will gradually become desynchronized from the actual audio playback position over time.

Single-User, Single-Gesture Limitation: The current implementation uses a single binary gesture (flex/relax cycle) for a single command (toggle play/pause). This requires multiple repeated gestures to navigate tracks (e.g., three flexes to skip two songs), which is somewhat cumbersome. A more sophisticated system would use distinct gesture patterns for distinct commands.

Environmental Sensitivity of Ultrasonic Sensor: The HC-SR04 sensor is sensitive to highly sound-absorbent materials (e.g., fabric, soft foam) and is not effective in environments with high levels of ambient ultrasonic noise. The volume control can also be inadvertently triggered by other objects passing within the sensor's detection cone.

6.3 FUTURE IMPROVEMENTS

Based on the critical evaluation above, the following improvements are proposed for future iterations of this system:

Wireless Architecture (Priority 1): The most impactful single improvement would be the integration of a Bluetooth module (e.g., HC-05) or a Wi-Fi module (e.g., ESP8266/ESP32) to replace the USB serial connection. This would eliminate the physical tether to the PC, greatly increasing user mobility. The ESP32 could also serve as a standalone microcontroller with sufficient processing power to handle both the sensor logic and wireless communication, potentially replacing the Arduino-PC combination entirely.

Wireless EMG Electrode Patch (Priority 2): Replacing the wired electrodes with a self-contained, wireless EMG patch worn on the forearm would transform the system from a laboratory prototype into a practical wearable device. Commercial EMG patches with Bluetooth Low Energy (BLE) interfaces are increasingly available.

Adaptive Threshold Calibration: Implementing a software-based auto-calibration routine would eliminate the need for manual trimpot adjustment. The system could monitor the baseline EMG signal for the first 5 seconds after power-on and automatically calculate appropriate flex and relax threshold values based on the user's resting signal level and a brief prompted contraction.

Dedicated RTC Module: Integrating a DS3231 Real-Time Clock module onto the I2C bus would provide accurate timekeeping independent of the PC connection, allowing the on-device clock to remain accurate even when the desktop application is not running.

Advanced Gesture Recognition: Implementing signal processing techniques such as Moving Average filtering, Root Mean Square (RMS) envelope extraction, or even a lightweight machine learning classifier (e.g., a Support Vector Machine trained on a small gesture dataset) could enable the recognition of multiple distinct gestures. For example: single flex = play/pause, double flex = next track, sustained flex = volume-boost mode. This would dramatically expand the command vocabulary of the system.

Expanded Song Metadata: Integrating an ID3 tag parser library for the DFPlayer would allow the system to display actual song title and artist name metadata read from the MP3 files, rather than relying on hardcoded song information in the firmware.

REFERENCES

[1] Reaz, M. B. I., Hussain, M. S., & Mohd-Yasin, F. (2006). Techniques of EMG signal analysis: Detection, processing, classification and applications. *Biological Procedures Online*, 8(1), 11–35. https://doi.org/10.1251/bpo115

[2] De Luca, C. J. (2002). Surface electromyography: Detection and recording. *Delsys Incorporated*, 10, 2011.

[3] Merletti, R., & Parker, P. (Eds.). (2004). *Electromyography: Physiology, engineering, and non-invasive applications*. Wiley-IEEE Press. https://doi.org/10.1002/0471678384

[4] Phinyomark, A., Limsakul, C., & Phukpattaranont, P. (2012). A novel feature extraction for robust EMG pattern recognition. *Journal of Computing*, 1(1), 71–80.

[5] Liu, W. (2025). Using Arduino Uno and EMG electrodes for electromyographic signal measurement and servo control. *Theoretical and Natural Science*, 98, 1–8. https://doi.org/10.54254/2753-8818/2025.21666

[6] Arduino. (2023). *Arduino UNO Rev3 documentation and technical reference*. Retrieved April 1, 2026, from https://www.arduino.cc/en/Main/ArduinoBoardUno

[7] Advancer Technologies. (n.d.). *Muscle sensor v3 user manual: Bio-signal hardware guidelines and application notes*. Advancer Technologies LLC.

[8] DFRobot. (2018). *DFPlayer Mini MP3 player for Arduino — Product wiki and datasheet*. Retrieved April 1, 2026, from https://wiki.dfrobot.com/DFPlayer_Mini_SKU_DFR0299

[9] Allegro MicroSystems. (2019). *PAM8403 — 3W stereo class-D audio amplifier datasheet*. Diodes Incorporated.

[10] Solomon Systech Limited. (2008). *SSD1306 advance information: 128×64 dot matrix OLED/PLED segment/common driver with controller*. Solomon Systech Limited.